

DBAD-4000 – Designing a Logical Database Gym & Fitness Centre

Group Number	Group 4 & Group 7
Team Members	1.Duy Pham 2. Christopher Findlay 3. Majd Alshahaf 4. Saihaj Mann 5. Vivek Patel
Modelling Tool	Lucidchart

Table of Contents

Design Process	3
First Entity Idea	3
Questions We Need to Discuss	3
Person / Member / Trainer	3
Trainer	4
Member	5
Class / Course / Schedule	5
Enrollment and Payment	6
Final Team Decisions	7
Main Entities	8
PERSON	8
MEMBER	8
TRAINER	8
FITNESS_CLASS	8
ENROLLMENT	8
PAYMENT	8
Final Business Rules	8
Final Entities and Attributes	9
Entity 1: PERSON (Supertype)	9
MEMBER (Subtype)	9
TRAINER (Subtype)	9
FITNESS_CLASS	10
ENROLLMENT	10
PAYMENT	10
Relationships, Cardinality and Participation	11
Specialization	11
Normalization to 3NF	11
Design Rationale	12
Data Dictionary	12
EERD	14
Team Contributions	15

Design Process

We reviewed the scenario and identified the main data the gym needs to manage: Members, trainers, classes and payments. We also found some details that are not explained in the scenario. The questions below are for our group to discuss before we make the final EERD.

Our final design needs 4 to 6 entities, 4 to 6 business rules, attributes, PK/FK, relationships, cardinality, participation, specialization and a short 3NF explanation.

First Entity Idea

Entity	Why we may need it	Keep?
PERSON	Common information for members and trainers	☒
MEMBER	Member information	☒
TRAINER	Trainer information	☒
FITNESS_CLASS	Class or scheduled class information	☒
ENROLLMENT	Connect members and classes	☒
PAYMENT	Member payments	☒

This was our initial design. We reviewed and updated it as a group.

Questions We Need to Discuss

Tick the choices we agree on and use the Notes column only when we need a short explanation. We do not need to turn every question into a business rule.

Person / Member / Trainer

Decide how MEMBER and TRAINER relate to PERSON and what common personal information should be stored.

MEMBER and TRAINER may share attributes such as name, email, and phone. We need to decide whether PERSON should be used as a supertype and whether the same person can have both roles. These decisions will affect our specialization and the attributes in the final entities.

Question / Decision	Yes	No	Notes / Choice	Reason
Can a trainer also be a member?	☐	☒	If Yes = Overlapping; if No = Disjoint	Trainers must complete all training courses before teaching, so their staff profile is separate from member accounts.
Should email be required?	☒	☐		Needed for login and sending receipts.
Should email be unique?	☒	☐		Prevents duplicate user accounts.
Should phone be required?	☒	☐		Needed for emergency contact.
Should phone be unique?	☐	☒	Family members may share a phone	Family members living in the same house might share a home phone.

Trainer

Decide what trainer information is needed and whether trainer qualifications should be included in the current database.

The scenario tells us that the gym has trainers, but it does not explain trainer skills or certificates. We should decide whether these details are important for our current scope. If one trainer can have multiple certificates, storing certificates may affect normalization and may require another entity.

Question / Decision	Yes	No	Notes / Choice	Reason
Can one trainer teach more than one type of class?	☒	☐		Experienced trainers can teach different class types.
Do we need to store trainer certificates?	☐	☒		Adding a certificate table exceeds our limit of 6 entities.
Should certificates have an expiry date?	☐	☒		Handled outside the database in future versions.
Must a required certificate be valid on the class date?	☐	☒		App-level logic, out of scope for this basic EERD.

Certificate design	—	—	☐ Include now ☒ Future requirement	Keeps the database simple and within the 6-entity limit.
--------------------	---	---	---	--

Member

Decide what information the gym actually needs to store about a member.

The scenario only says that the gym needs to manage members. It does not specify age requirements, emergency contacts, or waivers. The group should decide which information is necessary for our database and which requirements should stay outside the current scope.

Question / Decision	Yes	No	Notes / Choice	Reason
Should we store date of birth?	☒	☐		Required to check member age.
Is there a minimum age?	☒	☐	Age: 16	Members must be at least 16 to join the gym.
Do minors need parent / guardian approval?	☐	☒	Out of scope	Handled manually on paper at the front desk.
Does a member need an emergency contact?	☒	☐	In MEMBER	Stored as emergency phone directly inside MEMBER.
Does a member need to sign a waiver?	☒	☐	In MEMBER	Stored as a simple True/False flag in MEMBER.
Emergency contact design	—	—	☒ In MEMBER ☐ Separate entity ☐ Not in scope	Avoid creating another small entity.
Waiver design	—	—	☒ Status/date ☐ Document reference ☐ Not in scope	A simple True/False flag is enough.

Class / Course / Schedule

Define what FITNESS_CLASS represents before we design its attributes and relationships.

The word “class” can have different meanings. For example, “Yoga” may mean a type of class, while “Yoga on September 20 at 6:00 PM” is one scheduled session. We need to decide which meaning we will use for FITNESS_CLASS.

If FITNESS_CLASS represents one scheduled session, information such as date, start time, room, capacity, and trainer can belong to that entity. If it represents a course or class type with multiple sessions, we may need another entity such as CLASS_SESSION.

Question / Decision	Yes	No	Notes / Choice	Reason
Does FITNESS_CLASS represent one scheduled session?	☒	☐	If No, we may need COURSE + CLASS_SESSION	Each record is a real session with a specific date and time.
Can the same type of class have different start times?	☒	☐		Classes happen at different hours on different days.
Can the same type of class use different rooms?	☒	☐		Depends on room availability.
Should two classes be prevented from using the same room at the same time?	☒	☐		Prevents double-booking rooms.
Room conflict			☐ Business rule now ☒ Future requirement	Enforced by application code later.

Enrollment and Payment

Decide how members register for fitness classes and how payments are recorded.

A member may attend multiple fitness classes, and a fitness class may contain multiple members. We need to decide whether ENROLLMENT should be used to represent this relationship. We also need to define what PAYMENT represents and how it relates to a member.

Question / Decision	Yes	No	Notes / Choice	Reason
Do we need ENROLLMENT to connect MEMBER and FITNESS_CLASS?	☒	☐	Used to resolve M:N	Resolves the Many-to-Many (M:N) relationship.
Should duplicate enrollment in the same class be prevented?	☒	☐		A member cannot sign up for the exact same class twice.
Should enrollment stop when max capacity is reached?	☒	☐		Checked using max_capacity.

Can one member make many payments?	☒	☐		Members pay multiple times for monthly fees or classes.
Do we need to store payment method?	☒	☐		Stores Cash, Credit, or Debit for accounting.
What is a payment for?			☐ Membership ☐ Class ☒ Both ☐ Other	Covers general membership fees and extra class fees.

Final Team Decisions

After the discussion, summarize only the decisions that affect our database design.

Topic	Final Decision	Why / Design Impact
PERSON specialization	Total, Disjoint Subtyping (PERSON -> MEMBER / TRAINER)	We assume that each PERSON has only one role: MEMBER or TRAINER. This makes the specialization Total and Disjoint.
Trainer skills / certificates	Single text field specialization in TRAINER	Keeps the design simple and stays within our 6-entity limit.
Member information	Embed date_of_birth, emergency_phone, waiver_signed inside MEMBER	Meets safety and age requirements without extra tables.
FITNESS_CLASS meaning	Represents an actual scheduled class session	Stores session date, start time, room location, capacity, and assigned trainer.
Class time / room	Attributes in FITNESS_CLASS (location, start_time)	Avoids creating a separate ROOM entity for this lab.
Enrollment	Associative entity ENROLLMENT	Breaks the M:N relationship into two 1:M relationships.
Payment	Separate PAYMENT entity linked to MEMBER	Stores transaction history without duplicating data in MEMBER.

Main Entities

Based on our final team decisions, the database will use the following six main entities:

PERSON

Stores common information for both members and trainers, including first name, last name, email, and phone number. PERSON is the supertype of MEMBER and TRAINER.

MEMBER

Stores information specific to gym members, including date of birth, emergency phone number, and waiver status. MEMBER is a subtype of PERSON.

TRAINER

Stores information specific to gym trainers, including specialization and hourly rate. TRAINER is a subtype of PERSON.

FITNESS_CLASS

Represents a scheduled fitness class session. It stores the class name, date, start time, maximum capacity, location, and assigned trainer.

ENROLLMENT

Connects MEMBER and FITNESS_CLASS. It records which member enrolls in which fitness class and the enrollment date.

PAYMENT

Stores payment transactions made by members, including the payment amount, payment date, and payment method.

Final Business Rules

Rule	Final Business Rule
BR-01: Person Specialization	Every PERSON in the system must be either a MEMBER or a TRAINER, but not both. Therefore, the specialization is Total and Disjoint.
BR-02: Trainer Assignment	A TRAINER can teach multiple FITNESS_CLASS sessions, but each FITNESS_CLASS is led by exactly one TRAINER.
BR-03: Member Enrollment	A MEMBER can enroll in many FITNESS_CLASS sessions, and each FITNESS_CLASS can have many members.

BR-04: Enrollment Link	Each ENROLLMENT connects one MEMBER to one FITNESS_CLASS. The combination of member_id and class_id must be unique.
BR-05: Payment Tracking	A MEMBER can make multiple PAYMENT records over time, but each PAYMENT belongs to exactly one MEMBER.
BR-06: Member Rules	A MEMBER must be at least 16 years old and must have a signed waiver before enrolling in a class.

Final Entities and Attributes

Entity 1: PERSON (Supertype)

Attribute	Data Type	PK / FK	Required?	Validation / Notes
person_id	INT	PK	Yes	Auto-increment
first_name	VARCHAR(50)	None	Yes	NOT NULL
last_name	VARCHAR(50)	None	Yes	NOT NULL
email	VARCHAR(100)	None	Yes	UNIQUE, valid format
phone	VARCHAR(15)	None	Yes	Contact phone number
person_type	CHAR(1)	None	Yes	Discriminator: 'M' or 'T'

MEMBER (Subtype)

Attribute	Data Type	PK / FK	Required?	Validation / Notes
person_id	INT	PK, FK	Yes	FK to PERSON(person_id)
date_of_birth	DATE	None	Yes	Member must be >= 16 years old
emergency_phone	VARCHAR(15)	None	Yes	Emergency contact phone
waiver_signed	BOOLEAN	None	Yes	Default FALSE

TRAINER (Subtype)

Attribute	Data Type	PK / FK	Required?	Validation / Notes
person_id	INT	PK, FK	Yes	FK to PERSON(person_id)
specialization	VARCHAR(50)	None	No	Expertise (e.g., Yoga, Weightlifting)

hourly_rate	DECIMAL(6,2)	None	Yes	Must be > 0
-------------	--------------	------	-----	-------------

FITNESS_CLASS

Attribute	Data Type	PK / FK	Required?	Validation / Notes
class_id	INT	PK	Yes	Auto-increment
class_name	VARCHAR(50)	None	Yes	Name of the session
class_date	DATE	None	Yes	Scheduled date
start_time	TIME	None	Yes	Start time
max_capacity	INT	None	Yes	Must be > 0
location	VARCHAR(50)	None	Yes	Room name/number
trainer_id	INT	FK	Yes	FK to TRAINER(person_id)

ENROLLMENT

Attribute	Data Type	PK / FK	Required?	Validation / Notes
enrollment_id	INT	PK	Yes	Auto-increment
member_id	INT	FK	Yes	FK to MEMBER(person_id)
class_id	INT	FK	Yes	FK to FITNESS_CLASS(class_id) UNIQUE with member_id
enroll_date	DATE	None	Yes	Default current date

PAYMENT

Attribute	Data Type	PK / FK	Required?	Validation / Notes
payment_id	INT	PK	Yes	Auto-increment
amount	DECIMAL(7,2)	None	Yes	Must be > 0
payment_date	DATETIME	None	Yes	Date and time of payment
payment_method	VARCHAR(20)	None	Yes	'Credit', 'Debit', 'Cash'
member_id	INT	FK	Yes	FK to MEMBER(person_id)

Relationships, Cardinality and Participation

Relationship	Type	Cardinality	Participation	Notes
PERSON to MEMBER / TRAINER	Specialization	1:1	Total	Mandatory participation for supertype.
TRAINER leads FITNESS_CLASS	1:M	1 to Many	Mandatory on Class	One trainer can teach zero or many classes.
MEMBER creates ENROLLMENT	1:M	1 to Many	Optional on Member	Members can enroll in zero or many classes.
FITNESS_CLASS has ENROLLMENT	1:M	1 to Many	Optional on Class	A class can have zero to max-capacity enrollments.
MEMBER makes PAYMENT	1:M	1 to Many	Optional on Member	Members can make zero or many payments.

Specialization

Decision	Choose	Reason
Completeness	☒ Total ☐ Partial	Every person in the system must be registered as either a Member or a Trainer.
Disjoint	☒ Disjoint ☐ Overlapping	A person cannot be both at the same time.
Supertype / Subtypes	PERSON → MEMBER / TRAINER	We assume that a person can have only one role. A PERSON must be either a MEMBER or a TRAINER, but not both.

Normalization to 3NF

Design Decision	Our Explanation
Shared person data	Common attributes such as first_name, last_name, email, and phone are stored in PERSON. MEMBER and TRAINER only store attributes specific to their roles. This reduces duplicated data.

M:N relationship / ENROLLMENT	MEMBER and FITNESS_CLASS have a many-to-many relationship. ENROLLMENT resolves this relationship into two one-to-many relationships.
Multiple values (for example certificates)	Trainer specialization and class location are stored as simple attributes. Separate CERTIFICATE and ROOM entities are outside the scope.
Other 3NF decision	PAYMENT is stored separately because one MEMBER can make many payments. Payment details such as amount, date, and payment method depend on payment_id, which helps avoid duplicated payment data.

Design Rationale

Question	Team Explanation
Why did we choose these 6 entities?	PERSON, MEMBER, TRAINER, FITNESS_CLASS, ENROLLMENT, and PAYMENT cover all gym operations while matching the 4-6 entity requirement.
Why did we choose this specialization?	We use Total and Disjoint specialization because every PERSON must have one role, and we assume a person cannot be both a MEMBER and a TRAINER.
What important assumptions did we make?	We assume that emergency phone and waiver status can be stored directly in MEMBER because separate entities are not needed.
What did we leave outside the scope, and why?	Certificate expiration tracking tables and automatic room conflict checks were left outside the scope to prevent exceeding 6 entities.

Data Dictionary

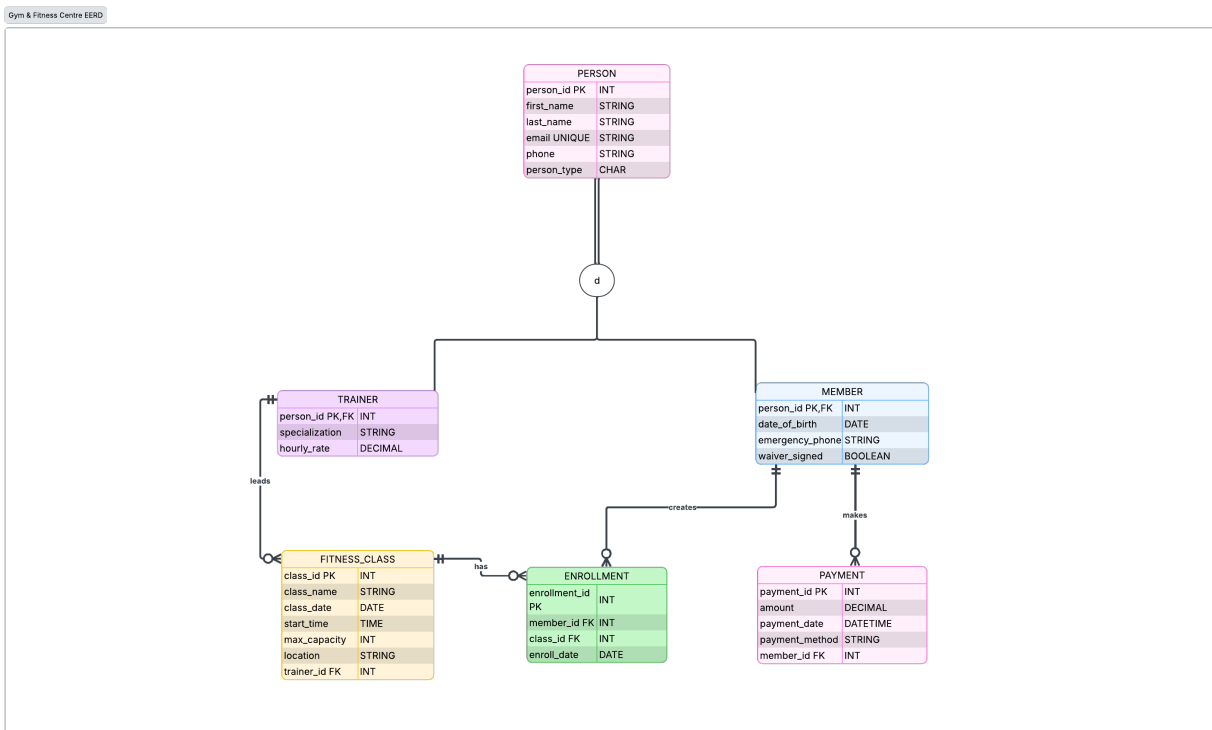
Entity	Attribute	Definition	Data Type	Key	Validation Rule
PERSON	person_id	Unique identifier for a person	INT	PK	Auto-increment
PERSON	first_name	First name	VARCHAR(50)	None	NOT NULL
PERSON	last_name	Last name	VARCHAR(50)	None	NOT NULL
PERSON	email	Contact email address	VARCHAR(100)	None	UNIQUE, valid email
PERSON	phone	Contact phone number	VARCHAR(15)	None	NOT NULL

PERSON	person_type	Specialization discriminator	CHAR(1)	None	'M' or 'T'
MEMBER	person_id	Foreign key referencing Person	INT	PK, FK	FK to PERSON(person_id)
MEMBER	date_of_birth	Member birth date	DATE	None	Age >= 16
MEMBER	emergency_phone	Emergency contact phone	VARCHAR(15)	None	NOT NULL
MEMBER	waiver_signed	Liability waiver agreement flag	BOOLEAN	None	Default FALSE
TRAINER	person_id	Foreign key referencing Person	INT	PK, FK	FK to PERSON(person_id)
TRAINER	specialization	Main fitness domain	VARCHAR(50)	None	Optional
TRAINER	hourly_rate	Hourly compensation rate	DECIMAL(6,2)	None	Must be > 0
FITNESS_CLASS	class_id	Unique ID for class session	INT	PK	Auto-increment
FITNESS_CLASS	class_name	Name of class session	VARCHAR(50)	None	NOT NULL
FITNESS_CLASS	class_date	Scheduled date	DATE	None	Required
FITNESS_CLASS	start_time	Scheduled start time	TIME	None	Required
FITNESS_CLASS	max_capacity	Max allowed participants	INT	None	Must be > 0
FITNESS_CLASS	location	Studio room identifier	VARCHAR(50)	None	NOT NULL
FITNESS_CLASS	trainer_id	Trainer leading class	INT	FK	FK to TRAINER(person_id)
ENROLLMENT	enrollment_id	Unique ID for enrollment	INT	PK	Auto-increment
ENROLLMENT	member_id	Member attending class	INT	FK	FK to MEMBER(person_id)

ENROLLMENT	class_id	Class session attended	INT	FK	FK to FITNESS_CLASS(class_s_id), UNIQUE with member_id
ENROLLMENT	enroll_date	Date of booking	DATE	None	Default current date
PAYMENT	payment_id	Unique transaction ID	INT	PK	Auto-increment
PAYMENT	amount	Amount paid in CAD	DECIMAL(7,2)	None	Must be > 0
PAYMENT	payment_date	Transaction timestamp	DATETIME	None	Default current timestamp
PAYMENT	payment_method	Payment method used	VARCHAR(20)	None	'Credit', 'Debit', 'Cash'
PAYMENT	member_id	Member making payment	INT	FK	FK to MEMBER(person_id)

EERD

Modelling tool: Lucidchart



Team Contributions

Team Member	Role	Specific Tasks
Chris Findlay	EERD Design	Created the EERD in Lucidchart. Worked with the team to check relationships, cardinalities, and Total/Disjoint specialization.
Duy Pham	Database Design & Documentation	Helped identify the main entities and attributes. Worked on the report and data dictionary. Joined the group discussion about business rules, relationships, and specialization.
Majd Alshahaf	Business Rules	Helped write and review the business rules. Joined the discussion about Member, Trainer, Class, Enrollment, and Payment.
Vivek	Relationships & Constraints	Helped check PK/FK, relationships, cardinalities, and participation. Also discussed the specialization between PERSON, MEMBER, and TRAINER.
Saihaj	Normalization & Review	Helped review 3NF, validation rules, and the data dictionary. Checked the final design with the team before submission.